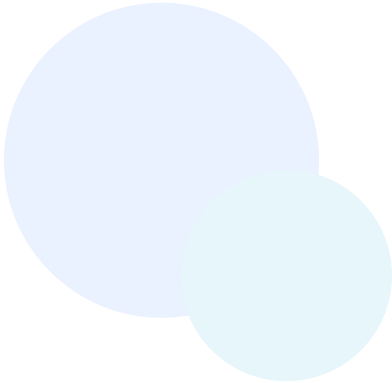




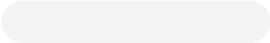
Reliability in Large Language Models

A literature review of definitions, evolving evaluation criteria, and evidence-based techniques for more dependable LLM outcomes

Central thesis

As LLMs have moved from narrow prediction systems to general assistants and tool-using agents, reliability has shifted from benchmark accuracy to end-to-end dependability: correctness, groundedness, robustness, calibration, safe behavior, and appropriate abstention.

Scope	Peer-reviewed research across NLP, ML, software engineering, medicine, law, and AI safety
Focus	Definitions, evolution of reliability, and practical techniques shown to improve outcomes
Date	June 10, 2026



Contents

Executive summary	3
1. Working definition	3
2. How the definition has evolved	3
3. Core dimensions of LLM reliability	5
4. Tools and techniques that improve reliability	5
5. Domain-specific reliability lessons	8
6. Evidence strength across techniques	9
7. Practical reliability architecture	9
8. Conclusion	10
Selected peer-reviewed references	10

Task vs. System

Executive summary

Chat required truthfulness due to instruction following finetuning?

The literature on LLM reliability has moved through a clear progression. Early NLP systems were judged largely by task-level scores; foundation models required broader capability and robustness evaluations; chat models forced researchers to address truthfulness, safety, calibration, and instruction following; and agentic, tool-using systems now require end-to-end reliability across retrieval, reasoning, tools, verification, and human oversight.

The most important practical conclusion is that reliable outcomes are engineered rather than assumed. The strongest evidence favors layered systems: retrieve authoritative evidence, use deterministic tools where possible, generate multiple candidate solutions for reasoning-heavy tasks, verify claims and code after generation, calibrate confidence, abstain when evidence is inadequate, and route high-stakes outputs to qualified humans.

Reliability question	How the literature answers it	Operational implication
What is reliability?	A task- and context-dependent property combining correctness, factuality, groundedness, robustness, calibration, safety, and abstention.	Define reliability separately for each use case and risk level.
Has the definition changed?	Yes. It has expanded from benchmark accuracy to system dependability as LLMs became general, interactive, and tool-using.	Evaluate model behavior across prompts, scenarios, tools, and user workflows.
What improves reliability most?	Grounding, execution, verification, uncertainty estimation, post-training, and human oversight each address different failure modes.	Use a layered architecture instead of relying on a single prompt or answer.

Bottom line

A high-scoring base model is not the same thing as a reliable LLM system. Reliability requires measurement, grounding, verification, controlled tool use, calibrated uncertainty, and domain-specific governance.

1. Working definition

✖

An LLM system is reliable for a given use case when it consistently produces outputs or actions that are correct enough, grounded in appropriate evidence, robust to reasonable input variation, safe and compliant, calibrated about uncertainty, and able to abstain, ask for clarification, use tools, or escalate when it cannot meet the required standard.

This definition deliberately treats reliability as conditional. A model can be reliable for spelling correction or tone rewriting while being unreliable for medical triage, legal research, financial modeling, or autonomous software modification. The acceptable error rate, required evidence, and escalation path differ by domain.

Normalise vs. Decode

It also treats reliability as a system property. The base model matters, but so do the prompt, retrieval corpus, tool interfaces, decoding settings, verification steps, logging, monitoring, and human review. HELM formalized this shift by evaluating language models across multiple scenarios and metrics rather than treating accuracy as a single sufficient score. TrustLLM extended the framing to trustworthiness dimensions such as truthfulness, safety, fairness, robustness, privacy, ethics, and misuse resistance (Liang et al., 2023; Huang et al., 2024).

A useful mental model

Reliability is the probability that the full LLM workflow will produce an acceptable answer or action under realistic conditions, including ambiguous instructions, distribution shift, bad retrieval, tool errors, and user misuse.

2. How the definition has evolved

The definition of LLM reliability has expanded alongside capability. The more general, interactive, and autonomous the systems became, the more reliability had to include factuality, uncertainty, safety, robustness, tool behavior, and governance.

Period	Dominant reliability concept	Representative literature	What changed
Narrow NLP systems	Task accuracy on fixed benchmarks: exact match, F1, BLEU, ROUGE, or classification accuracy.	Pre-LLM benchmark tradition; task-specific NLP evaluation.	Reliability was mostly distribution-bound performance.
Foundation models	Broad generalization across subjects and tasks; understanding capability boundaries.	MMLU, BIG-bench, BIG-Bench Hard, HELM (Hendrycks et al., 2021; Srivastava et al., 2023; Suzgun et al., 2023; Liang et al., 2023).	One model could do many things, so single-task accuracy became inadequate.
Truthfulness and hallucination	Factuality, faithfulness to evidence, and claim-level support.	TruthfulQA, hallucination surveys, FActScore, RARR (Lin et al., 2022; Ji et al., 2023; Min et al., 2023; Gao et al., 2023).	Fluent language was shown to be compatible with plausible falsehoods.
Instruction-following assistants	Alignment with user intent, helpfulness, harmlessness, refusal behavior, and calibration.	InstructGPT, TrustLLM, uncertainty studies, RLHF and DPO (Ouyang et al., 2022; Huang et al., 2024; Xiong et al., 2024; Rafailov et al., 2023).	Reliability became a question of interaction, not just prediction.
Tool-using agents	End-to-end dependability across reasoning, actions, retrieval, APIs, execution, and monitoring.	RAG, ReAct, Toolformer, PAL, CodeT, AlphaCode (Lewis et al., 2020; Yao et al., 2023; Schick et al., 2023; Gao et al., 2023; Chen et al., 2023; Li et al., 2022).	The model became one component in a larger socio-technical system.

2.1 From task accuracy to capability boundaries

Early NLP reliability was measured by performance on a narrow distribution. This became insufficient when foundation models began to show broad transfer. MMLU framed reliability as performance across dozens of subjects; BIG-bench expanded the task scope even further; HELM argued that evaluation had to be holistic, covering accuracy, robustness, fairness, bias, toxicity, calibration, and efficiency (Hendrycks et al., 2021; Srivastava et al., 2023; Liang et al., 2023).

This phase changed the research question from 'How accurate is the model on this task?' to 'Where does this general model work, where does it fail, and how stable are those results across prompts, distributions, and metrics?'

2.2 From general ability to factual reliability

As LLMs were used for open-ended answers, the central failure mode became the hallucination: a fluent answer containing false or unsupported claims. TruthfulQA showed that models could reproduce common human falsehoods; hallucination surveys clarified that errors differ across summarization, QA, dialogue, translation, and data-to-text generation; and FActScore showed why long-form reliability needs atomic claim-level evaluation rather than a single paragraph-level judgment (Lin et al., 2022; Ji et al., 2023; Min et al., 2023).

2.3 From answers to aligned interaction

Instruction tuning and human feedback broadened reliability again. The model now had to follow user intent, avoid harmful content, refuse unsafe requests, and remain useful without overclaiming. InstructGPT showed that human preference training could make models more helpful and truthful relative to base models, but also made clear that alignment is not a guarantee of factual correctness (Ouyang et al., 2022). *More truthful?*

2.4 From model outputs to system outcomes

RAG, code execution, tool use, and agents turned reliability into an end-to-end systems problem. ReAct interleaves reasoning and external actions, while Toolformer studies how models can learn API use. These methods improve reliability by letting models access external information or deterministic computation, but they also introduce new failure modes such as bad tool choice, stale retrieval, prompt injection, and cascading multi-step errors (Yao et al., 2023; Schick et al., 2023).



3. Core dimensions of LLM reliability

The literature now treats reliability as multi-dimensional. The table below translates common research terms into operational questions a deployment team can test.

Dimension	Operational question	Typical failure mode	Evaluation signal
Task correctness	Does the output solve the user's actual problem?	Wrong answer, incomplete reasoning, wrong format.	Task score, expert grading, unit tests, user success.
Factuality	Are the claims true relative to the world or domain source?	Hallucinated facts, fabricated citations, outdated claims.	Fact-checking, FActScore-style atomic support.
Faithfulness	Is the answer supported by the supplied or retrieved context?	Summary adds unsupported facts; RAG answer ignores documents.	Entailment checks, citation validation, source overlap.
Calibration	Does confidence track likelihood of correctness?	Confident false answer; unnecessary hedging on known facts.	Expected calibration error, Brier score, failure prediction.
Abstention	Does the system decline, ask, or escalate when it should?	Guessing outside scope; over-refusing benign prompts.	Selective risk, abstention precision/recall, escalation audits.
Robustness	Do results hold under paraphrases, noise, or distribution shift?	Answer flips under equivalent wording or formatting.	Prompt ensembles, adversarial tests, perturbation suites.
Consistency	Do similar inputs produce compatible answers?	Contradictory answers across runs or sessions.	Self-consistency, invariance testing, regression tests.
Safety and compliance	Does the system avoid harmful, disallowed, or privacy-violating outputs?	Unsafe instructions, leakage, policy violations.	Red-team tests, safety benchmarks, policy audits.
Fairness	Are errors, refusals, and tone uneven across groups?	Stereotyping, disparate refusal, unequal error rates.	Subgroup evaluation, bias benchmarks, human review.
Operational reliability	Does the full workflow survive tool, latency, and monitoring failures?	Silent tool failure, stale index, broken fallback.	Observability, uptime, tool-call audits, rollback tests.

Prompt sensitivity matters

A single prompt score can mislead. Reliability evaluation should report ranges across prompt templates, formats, decoding settings, and retrieval conditions, especially when model ranking or deployment decisions depend on small differences.

4. Tools and techniques that improve reliability

No single technique solves reliability. The best-supported interventions each address a different failure mode: retrieval addresses stale or missing knowledge, execution addresses arithmetic and code errors, reasoning scaffolds address multi-step problem solving, verification addresses unsupported claims, calibration addresses overconfidence, and post-training addresses instruction following and safety.

4.1 Multi-metric, multi-prompt evaluation

The first reliability tool is disciplined measurement. HELM and TrustLLM are influential because they reject a single-score view of reliability. They evaluate across tasks and dimensions such as accuracy, robustness, calibration, fairness, toxicity, privacy, and ethics (Liang et al., 2023; Huang et al., 2024).

Proven pattern

Use evaluation suites that include in-domain tasks, out-of-domain tasks, paraphrases, adversarial examples, multi-prompt runs, subgroup analysis, calibration, and abstention metrics. Evaluation alone does not fix failures, but it prevents false confidence.

4.2 Retrieval-augmented generation and evidence grounding

Retrieval-augmented generation improves reliability by giving the model relevant external evidence at answer time. The original RAG paper combined parametric model memory with non-parametric retrieved memory and improved knowledge-intensive QA. Later work showed that retrieval must be handled carefully: irrelevant or misleading context can degrade answers, so reranking, filtering, attribution, and robustness training matter (Lewis et al., 2020; Yoran et al., 2024).

RAG variant	Reliability benefit	Best suited domains	Limits
Standard RAG	Reduces reliance on memorized facts and enables source-grounded answers.	Enterprise QA, scientific QA, product support, policy search.	Fails when retrieval misses, returns stale documents, or supplies misleading context.
Iterative or active retrieval	Retrieves during generation when the model recognizes an information need.	Long-form answers, multi-hop research, evolving topics.	More tool calls increase latency and create more opportunities for bad retrieval.
Attribution and revision	Checks whether claims are supported and edits unsupported content.	Research synthesis, biographies, legal and medical drafts.	Attribution tools can still miss subtle contradictions.
Robust RAG	Filters irrelevant context and trains systems to resist bad retrieval.	Open-domain QA, web search, enterprise search.	Requires realistic negative examples and monitoring.
Knowledge graph grounding	Adds structured entity and relationship constraints.	Biomedical, legal, financial, and technical domains.	Graph quality and freshness become reliability bottlenecks.

Practical rule

RAG improves reliability only when the system can answer: What source supports this claim? Is the source authoritative? Is it current? Did the citation actually support the sentence?

4.3 Reasoning scaffolds: decomposition, search, and self-consistency

For reasoning-heavy tasks, reliability improves when the model decomposes the problem instead of producing an immediate answer. Chain-of-thought prompting improved arithmetic, symbolic, and commonsense reasoning in large models. Self-consistency improved reliability further by sampling multiple reasoning paths and selecting the most consistent answer. Least-to-most prompting and Tree of Thoughts extend this idea by decomposing tasks or searching over possible reasoning paths (Wei et al., 2022; Wang et al., 2023; Zhou et al., 2023; Yao et al., 2023).

These methods are strongest when the task has a stable answer and the reasoning path can be checked. They are weaker for factual claims without evidence because a model can generate a coherent explanation for a false answer.

Reliable pattern

Use reasoning scaffolds for multi-step reasoning, but pair them with retrieval or execution when correctness depends on external facts, arithmetic, code, law, medicine, or finance.

4.4 Tool use and deterministic execution

Tool use improves reliability by shifting part of the task from probabilistic text generation to deterministic or externally verifiable computation. Program-Aided Language Models and Program of Thoughts ask the model to write executable code, then rely on an interpreter for arithmetic, symbolic manipulation, and data analysis. In software engineering, systems such as AlphaCode and CodeT use execution and tests to filter or select generated programs (Gao et al., 2023; Chen et al., 2023; Li et al., 2022).

Tool or technique	Reliability mechanism	Best uses	Primary risk
Calculator or Python	Delegates arithmetic and symbolic work to execution.	Math, finance, statistics, spreadsheets, analytics.	The generated formula or code can be wrong.
Unit tests	Rejects code that does not satisfy executable examples.	Code generation, SQL, scripts, data pipelines.	Tests may be incomplete or generated from the same mistaken assumption.

Tool or technique	Reliability mechanism	Best uses	Primary risk
Static analysis and type checking	Detects structural code errors before runtime.	Software engineering and automation.	Does not prove semantic correctness.
Search or database APIs	Connects the model to current or proprietary data.	Research, enterprise QA, customer support.	Wrong query, stale data, or unsafe retrieved content.
Structured tool schemas	Constrains actions and arguments.	Agents, operations, workflow automation.	Tool misuse can still cascade across steps.

Engineering control

Sandbox tool execution, validate tool arguments, log every tool call, retry transient failures, and require explicit fallback behavior when a tool returns low-confidence or inconsistent results.

4.5 Verification, fact checking, and hallucination detection

A recurring reliability pattern is to separate generation from verification. SelfCheckGPT detects hallucinations by sampling multiple outputs and checking semantic consistency. Chain-of-Verification asks the model to draft, generate verification questions, answer them independently, and revise. FactScore evaluates long-form answers by decomposing them into atomic facts. Semantic entropy estimates uncertainty over meanings rather than surface strings (Manakul et al., 2023; Dhuliawala et al., 2024; Min et al., 2023; Farquhar et al., 2024).

Verifier	What it catches	Best domains	Limit
Claim extraction + retrieval	Unsupported or contradicted factual claims.	Research, legal, medical, enterprise QA.	Needs authoritative sources and good entailment checks.
Sampling consistency	Unstable facts that vary across generated answers.	Biographies, open-ended QA, summaries.	Consistent wrong answers can still pass.
Executable verification	Arithmetic, code, SQL, and data errors.	Software, analytics, finance.	Execution validates the program, not the user's intent.
Independent model critique	Format errors, omissions, obvious contradictions.	Drafting and review workflows.	Verifier can share generator blind spots.
Expert review	Domain-specific errors and risk judgments.	Medicine, law, finance, safety.	Costly and slower, but necessary in high-stakes domains.

4.6 Calibration, uncertainty, and abstention

Reliability requires knowing when not to answer. Uncertainty work shows that verbal confidence is often overconfident, though calibration and failure prediction tend to improve with model capability and with methods such as sampling, aggregation, and carefully designed confidence prompts. Abstention is increasingly treated as a reliability mechanism rather than a deficiency: a system should decline, ask for missing information, or escalate when evidence is weak or the task is unsafe (Xiong et al., 2024).

Selective answering

For high-risk tasks, use thresholds that combine retrieval relevance, source authority, verifier score, self-consistency, tool success, and domain risk. Below threshold, the reliable behavior is to abstain, ask for clarification, or escalate.

4.7 Alignment and post-training

Post-training improves reliability by shaping model behavior toward human preferences, safety policies, and instruction-following norms. RLHF made instruction-following assistants more helpful and often more truthful relative to base models. Direct Preference Optimization simplified preference optimization, while fine-grained RLHF supplies more detailed feedback about factuality, relevance, and completeness in long outputs (Ouyang et al., 2022; Rafailov et al., 2023; Wu et al., 2023).

The limitation is important: alignment can improve usefulness and reduce harmful behavior, but it does not guarantee truth. A model can be polite, aligned, and still factually wrong. Post-training is therefore best seen as one layer in a

reliability stack.

4.8 Human-in-the-loop oversight

The literature increasingly supports human oversight not as a cosmetic safeguard but as a design requirement for high-stakes systems. Human review is most valuable when the LLM provides traceable evidence, uncertainty signals, tool logs, and clearly separated claims, rather than a polished answer with no audit trail.

Oversight principle
The more consequential the decision, the more the LLM should act as a drafting, retrieval, analysis, or triage assistant - not as the final decision authority.

5. Domain-specific reliability lessons

Reliability techniques do not transfer uniformly. Each domain has characteristic failure costs and evidence standards.

Domain	Dominant reliability risks	Most useful techniques	Human role
Healthcare	Confident wrong guidance, outdated guidelines, missing patient context, unsafe triage.	Guideline RAG, source quoting, contraindication checks, uncertainty thresholds, clinician review.	Clinician remains accountable; LLM supports retrieval, summarization, and drafting.
Law	Invented cases, wrong jurisdiction, stale law, false assumptions in prompts.	Authoritative legal retrieval, jurisdiction/date checks, quote-level citation validation, lawyer review.	Lawyer validates legal reasoning and applicability.
Software	Compiling but semantically wrong code, security bugs, dependency errors.	Execution, unit tests, static analysis, sandboxing, code review, rollback.	Engineer reviews architecture, security, maintainability.
Math and finance	Arithmetic mistakes, wrong formulas, unit errors, overconfident forecasts.	Python/calculators, spreadsheet validation, units checks, formula transparency, audit logs.	Analyst validates assumptions and model scope.
Long-form research	Unsupported claims, selective citation, contradictions across sections.	RAG, atomic fact checks, attribution, contradiction detection, citation audit.	Researcher evaluates source quality and synthesis.
Enterprise operations	Wrong policy interpretation, stale knowledge base, unsafe workflow actions.	Restricted tools, access control, retrieval freshness, action confirmation, monitoring.	Operator approves high-impact actions and handles exceptions.

5.1 Healthcare

Healthcare reliability requires factual accuracy, guideline grounding, appropriate uncertainty, and clinician oversight. Retrieval helps because medical knowledge is specialized and changes, but retrieval does not remove the need to validate applicability to the patient. The safest role for LLMs is usually support: summarizing records, retrieving guidelines, drafting patient-facing explanations, or helping clinicians reason through documentation.

5.2 Law

Legal reliability is difficult because small factual errors can change outcomes. Legal research systems must verify jurisdiction, date, authority, and whether cited cases or statutes actually support the proposition. The model should not invent legal authority or accept false assumptions in a user's prompt. For legal work, source-level traceability and lawyer review are core reliability controls.

5.3 Software engineering

Software is one of the domains where reliability can be improved most concretely, because outputs can be compiled, tested, linted, scanned, and executed. The strongest pattern is generate, test, repair, and review. A model's verbal confidence matters less than whether code passes meaningful tests and satisfies design and security requirements.

5.4 Numerical, financial, and analytic work

For numerical tasks, the model should not be trusted to do arithmetic in prose. Program-aided reasoning, calculators, spreadsheet formulas, unit checks, and reproducible notebooks shift reliability from plausible explanation to inspectable computation. The remaining risk is not arithmetic alone; it is whether the model encoded the right assumption, formula, or dataset.

5.5 Long-form writing and research synthesis

Long-form reliability is claim-level. A paragraph can be mostly correct while containing one serious false claim. Methods such as FActScore, attribution revision, SelfCheckGPT, and Chain-of-Verification are especially relevant because they expose unsupported claims, contradictions, and unstable facts.

6. Evidence strength across techniques

The following synthesis rates evidence strength by breadth of empirical support, reproducibility across tasks, and clarity of the reliability mechanism. These are not universal rankings; they are deployment heuristics.

Technique	Evidence strength	Most reliable use cases	Main caveat
RAG with strong retrieval and citation checks	High	Knowledge-intensive QA, enterprise search, legal and medical drafting.	Bad retrieval or unsupported citations can make answers worse.
Program execution and calculators	High	Math, finance, spreadsheets, statistics, data analysis.	The model may write the wrong program.
Unit tests and execution filtering	High	Code generation, SQL, scripts, data transformation.	Tests may be incomplete or brittle.
Self-consistency and multi-sampling	High for reasoning benchmarks	Math, symbolic reasoning, multi-step QA.	Can converge on a common wrong answer.
Decomposition and search over reasoning paths	Moderate to high	Planning, compositional tasks, tutoring, puzzles.	Less reliable for factual claims without evidence.
Claim verification and hallucination detection	Moderate to high	Long-form QA, summaries, research synthesis.	Verifier may share generator blind spots.
Calibration and abstention	Necessary but still developing	Medicine, law, compliance, finance, safety.	Open-ended text correctness is hard to calibrate.
RLHF, DPO, and preference tuning	High for behavior shaping	Instruction following, tone, refusal behavior, safety.	Does not guarantee factual truth.
Human expert review	Essential in high-stakes domains	Clinical, legal, financial, safety-critical decisions.	Cost and latency limit scalability.

7. Practical reliability architecture

A production-grade reliability architecture should be built as a sequence of controls. Each control should have a measurable acceptance criterion and a defined fallback.

- Define the operating domain: allowed tasks, users, sources, risk level, acceptable error rate, and escalation path.
- Evaluate before deployment: use domain benchmarks, adversarial tests, prompt variation, subgroup analysis, calibration, and human expert review.
- Ground factual answers: retrieve from authoritative sources, rerank, check freshness, and require source support for claims.
- Use tools where correctness is externally checkable: calculators, Python, SQL, compilers, static analyzers, theorem provers, and domain APIs.
- Separate generation from verification: extract claims, retrieve evidence, check entailment, detect contradictions, and revise unsupported statements.

- Estimate uncertainty and abstain: combine retrieval confidence, verifier confidence, self-consistency, tool success, and domain risk.
- Log and monitor: store prompts, retrieved documents, tool calls, model versions, verifier scores, user corrections, and incidents.
- Keep humans in the loop for high-stakes decisions: expert review should be triggered by risk, uncertainty, missing evidence, or irreversible action.

Deployment test

Before launch, ask whether the system can identify when it should not answer. A system that cannot abstain or escalate is usually not reliable enough for high-stakes use.

8. Conclusion

The meaning of reliability in LLMs has broadened as the systems have become more capable. In early NLP, reliability mostly meant benchmark accuracy. For foundation models, it meant broad generalization and capability mapping. For chat models, it added truthfulness, calibration, harmlessness, and instruction following. For RAG and agents, it now means end-to-end dependability across retrieval, tools, reasoning, verification, uncertainty, and human oversight.

The clearest lesson from the literature is that reliable LLM outcomes are engineered. The most dependable systems do not rely on a single model answer. They ground claims in evidence, execute computations, test generated code, sample and verify alternatives, calibrate uncertainty, abstain when needed, and involve qualified humans where the cost of error is high.

Final takeaway

Reliability is not a property a model simply has. It is a property a system earns through careful scoping, measurement, grounding, verification, calibrated abstention, and governance.

Selected peer-reviewed references

This selected bibliography emphasizes peer-reviewed conference, journal, and proceedings papers most directly connected to LLM reliability, evaluation, hallucination, tool use, and trustworthy deployment.

- Chen, B. et al. (2023). CodeT: Code Generation with Generated Tests. International Conference on Learning Representations.
- Dhuliawala, S. et al. (2024). Chain-of-Verification Reduces Hallucination in Large Language Models. Findings of the Association for Computational Linguistics: ACL.
- Farquhar, S., Kossen, J., Kuhn, L., & Gal, Y. (2024). Detecting hallucinations in large language models using semantic entropy. Nature.
- Gao, L. et al. (2023). RARR: Researching and Revising What Language Models Say, Using Language Models. Association for Computational Linguistics.
- Gao, L. et al. (2023). PAL: Program-Aided Language Models. International Conference on Machine Learning.
- Hendrycks, D. et al. (2021). Measuring Massive Multitask Language Understanding. International Conference on Learning Representations.
- Huang, Y. et al. (2024). TrustLLM: Trustworthiness in Large Language Models. International Conference on Machine Learning, PMLR 235.
- Ji, Z. et al. (2023). Survey of Hallucination in Natural Language Generation. ACM Computing Surveys. doi:10.1145/3571730.
- Jiang, Z. et al. (2023). Active Retrieval Augmented Generation. Empirical Methods in Natural Language Processing.
- Lewis, P. et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. Advances in Neural Information Processing Systems.
- Li, Y. et al. (2022). Competition-Level Code Generation with AlphaCode. Science. doi:10.1126/science.abq1158.
- Liang, P. et al. (2023). Holistic Evaluation of Language Models. Transactions on Machine Learning Research.
- Lin, S., Hilton, J., & Evans, O. (2022). TruthfulQA: Measuring How Models Mimic Human Falsehoods. Association for Computational Linguistics.
- Manakul, P., Liusie, A., & Gales, M. J. F. (2023). SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection for Generative Large Language Models. Empirical Methods in Natural Language Processing.
- Min, S. et al. (2023). FActScore: Fine-grained Atomic Evaluation of Factual Precision in Long Form Text Generation. Empirical Methods in Natural Language Processing.
- Ouyang, L. et al. (2022). Training language models to follow instructions with human feedback. Advances in Neural Information Processing Systems.

- Rafailov, R. et al. (2023). Direct Preference Optimization: Your Language Model is Secretly a Reward Model. *Advances in Neural Information Processing Systems*.
- Schick, T. et al. (2023). Toolformer: Language Models Can Teach Themselves to Use Tools. *Advances in Neural Information Processing Systems*.
- Srivastava, A. et al. (2023). Beyond the Imitation Game: Quantifying and extrapolating the capabilities of language models. *Transactions on Machine Learning Research*.
- Suzgun, M. et al. (2023). Challenging BIG-Bench Tasks and Whether Chain-of-Thought Can Solve Them. *Findings of the Association for Computational Linguistics: ACL*.
- Wang, X. et al. (2023). Self-Consistency Improves Chain of Thought Reasoning in Language Models. *International Conference on Learning Representations*.
- Wei, J. et al. (2022). Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. *Advances in Neural Information Processing Systems*.
- Wu, Z. et al. (2023). Fine-Grained Human Feedback Gives Better Rewards for Language Model Training. *Advances in Neural Information Processing Systems*.
- Xiong, M. et al. (2024). Can LLMs Express Their Uncertainty? An Empirical Evaluation of Confidence Elicitation in LLMs. *International Conference on Learning Representations*.
- Yao, S. et al. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. *International Conference on Learning Representations*.
- Yao, S. et al. (2023). Tree of Thoughts: Deliberate Problem Solving with Large Language Models. *Advances in Neural Information Processing Systems*.
- Yoran, O. et al. (2024). Making Retrieval-Augmented Language Models Robust to Irrelevant Context. *International Conference on Learning Representations*.
- Zhou, D. et al. (2023). Least-to-Most Prompting Enables Complex Reasoning in Large Language Models. *International Conference on Learning Representations*.